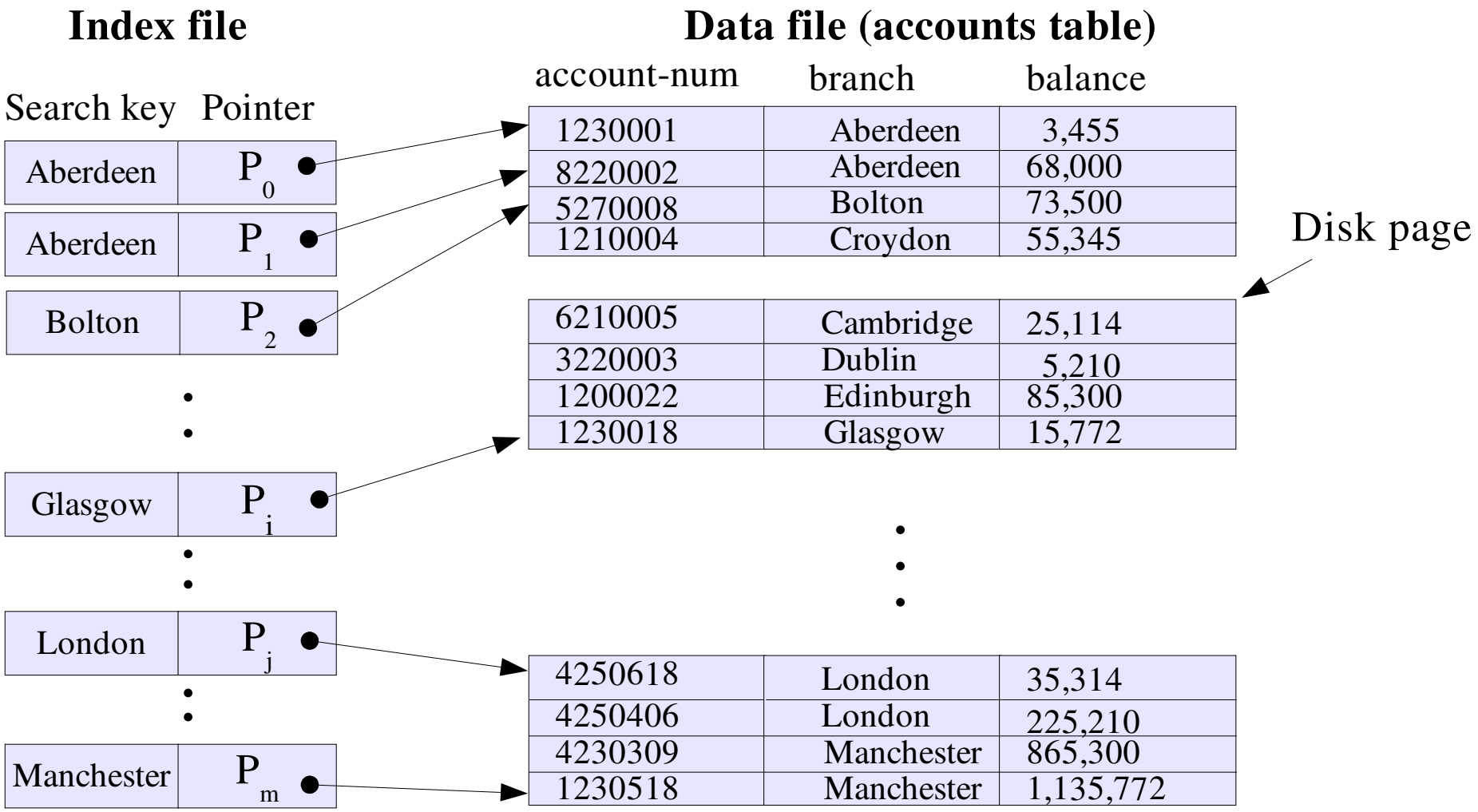
More on indexing: B+ trees

Outline

- Motivation: Search example
  - Cost of searching with and without indices
- B+ trees
  - Definition and structure
- B+ tree operations
  - Inserting
  - Deleting

Dense ordered index on accounts



Index access – performance

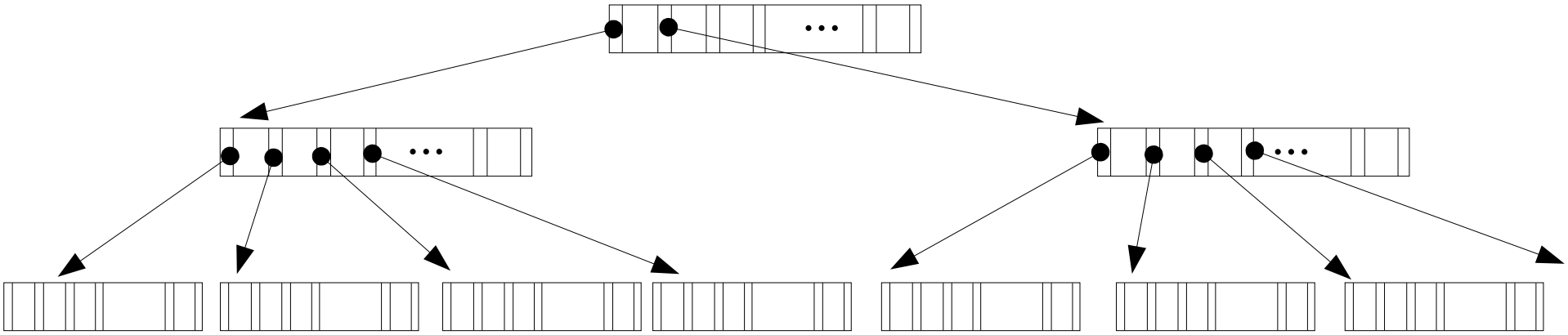
- Assume
  - N_R : number of records in the table
  - F_R : data blocking factor, # of data records that fit in a block
  - F_i : index blocking factor, # of index entries that fit in a block
- Cost of searching for a data record (in disk I/Os)
  - Assume a dense primary index (#records=#index entries)
  - Binary search on index, then fetch data block
    - Cost = $1 + \lceil \log_2(N_R / F_i) \rceil$
    - Assuming index is stored as a sequential file of blocks
      - Note that this makes updating the index expensive

Index access – Example

- N_R : number of records in the table
  - 2^{20} (over 1 million records)
- F_R : data blocking factor, # of data records that fit in a block
  - 10 (400 byte records, and 4KB pages)
- F_i : index blocking factor, # of index entries that fit in a block
  - 256 = 2^8 (16 byte *search-key,pointer* pairs in 4KB pages)
- Cost of search (without an index)
  - $N_R / F_R = 2^{20} / 10 = 104858$ da (disk accesses, ~ 18 minutes)
- Cost of search (using binary search of ordered index)
  - $1 + \lceil \log_2(N_R / F_i) \rceil = 1 + \log_2(2^{20}/2^8) = 13$ da (130 msec)

(Calculations assume 10msec per disk access)

Tree-structured indices

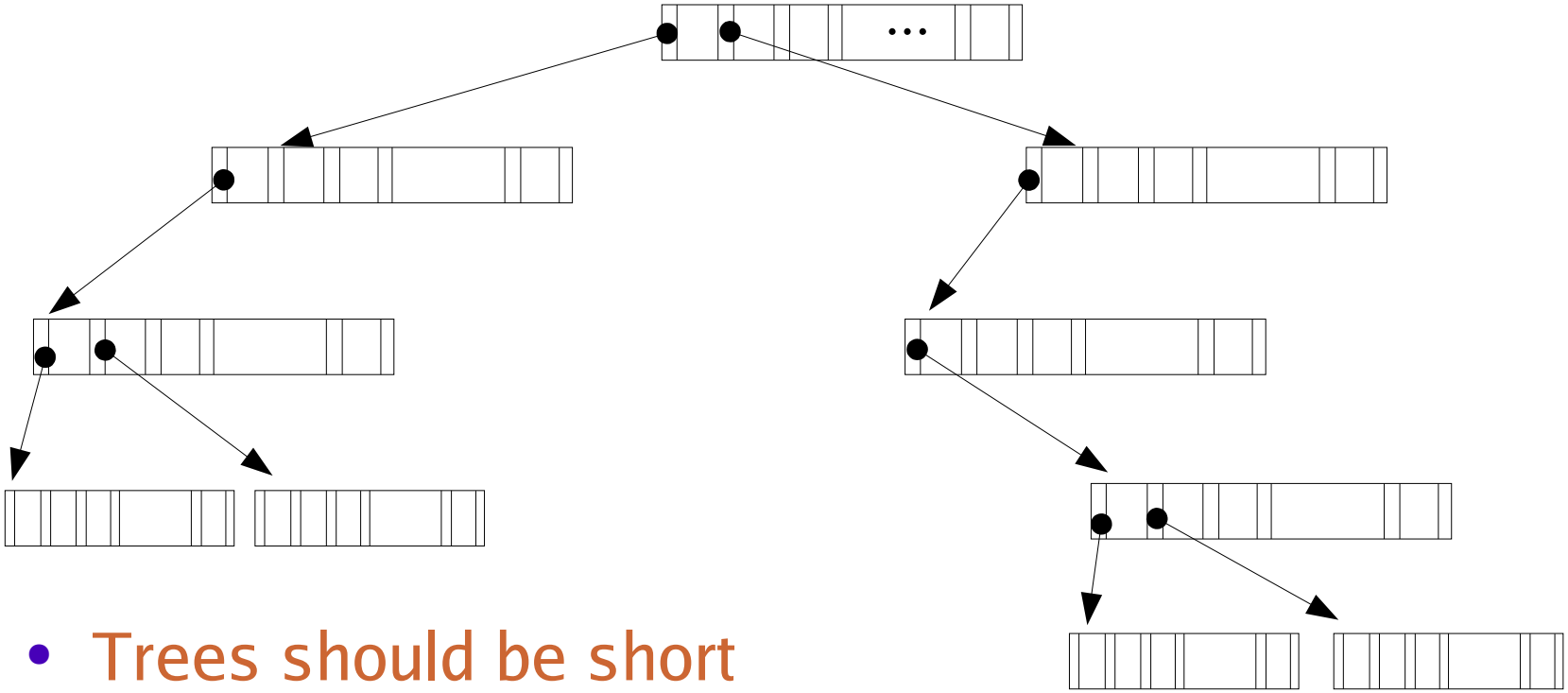


- Each tree node is stored in a single disk block
- Each node packs in a large number of key,pointer pairs
  - Number of children of a node is called *fan-out (m)*
  - Trees Reduce search cost significantly
    - Best case search cost: $1 + \lceil \log_m (N_R / F_i) \rceil$

Tree Index access – performance

- Example revisited
  - N_R : number of records in the table
    - 2^{20} (over 1 million records)
  - F_R : record blocking factor, # of data records that fit in a block
    - 10 (400 byte records, and 4KB pages)
  - F_i : index blocking factor, # of index entries that fit in a block
    - 256 = 2^8 (16 byte key,pointer pairs in 4KB pages)
  - Assume all nodes in the tree are half-full (fan out $m = F_i/2 = 128$)
    - $1 + \lceil \log_m (N_R / F_i) \rceil$
    - $1 + \lceil \log_{128} (2^{20}/2^8) \rceil = 1 + \lceil \log_{128} (2^{12}) \rceil$ = 3 da (30msec)

Not all trees are good trees



- Trees should be short
  - Nodes should be relatively full
  - Tree should be balanced

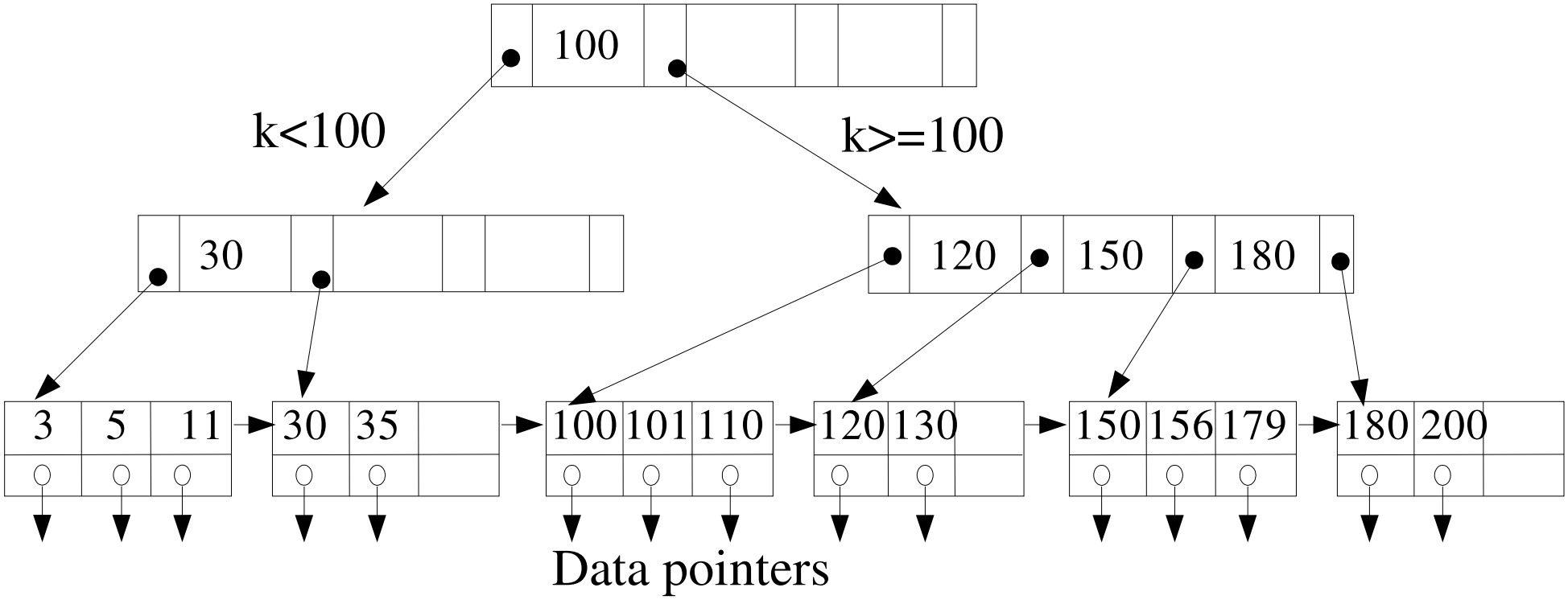
B trees

- General form of multi-level index
- Generalise binary search trees
- Balanced tree
  - All leaves are at same depth
- Efficient insert and delete
  - At the expense of some space overhead

B+ tree

- Popular variant of the B tree
  - B tree
    - Data pointers may be stored in internal nodes
    - Every value of the search field appears once
      - at some level in the tree
  - B+ tree
    - Data pointers stored only in leaf nodes
    - Internal nodes contain only keys and tree pointers
      - Can pack more pointers in internal nodes
      - Improved search time due to fewer levels in the tree
      - No wasted space due to “null” tree pointers in leaf nodes

B+ tree



- Internal node of order p=4, leaf nodes of order 3
  - i.e. Internal/leaf nodes must have between 2 and 4 pointers
- Leaf nodes are chained using sequence pointers

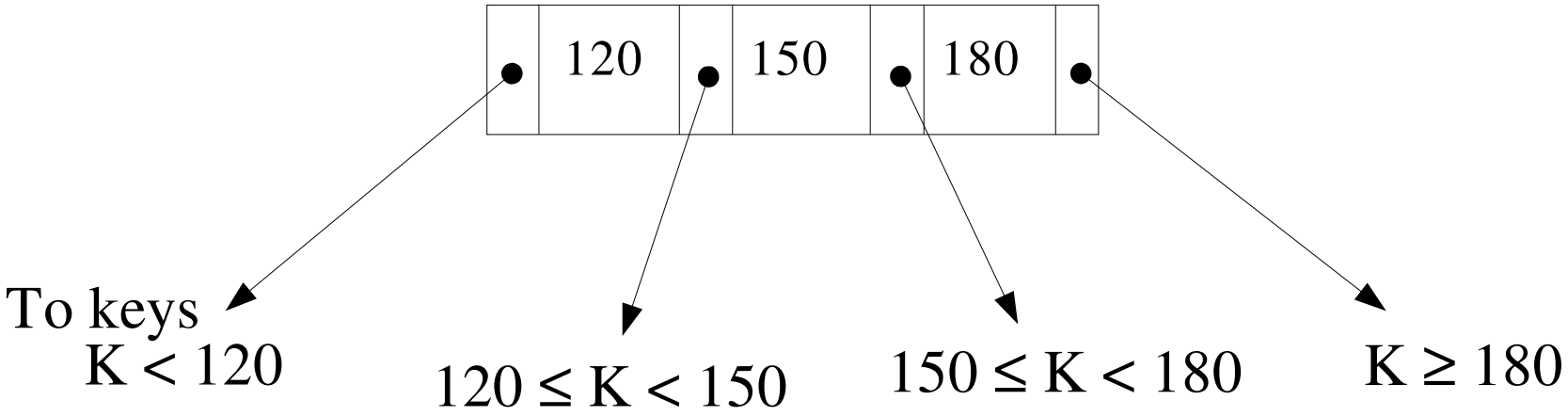
B+ tree – internal nodes

- Each internal node of B tree is of the form
  - $<P_1, K_1, P_2, K_2, ..., P_{q-1}, K_{q-1}, P_q>$
    - Where q <=p : The B+ tree is said to be of **order p**
    - Order of internal node ≠ Order of leaf node
  - P_i is a tree pointer, points to another node in the B+ tree
- Within each internal node, $K_1 < K_2 < .. < K_{q-1}$
  - For all key field values X in the subtree pointed to by P_i
    - $K_{i-1} <= X < K_i$ for $1<i<q$
    - $X < K_i$ for i=1
    - $K_{i-1} <= X$ for i=q

B+ tree – internal nodes (continued)

- Each internal node has at most p pointers
- Each internal node has at least $\lceil p/2 \rceil$ pointers
  - Except for the root
  - Root node has at least two pointers unless it is a leaf node
- Internal node with q pointers has q-1 search field values

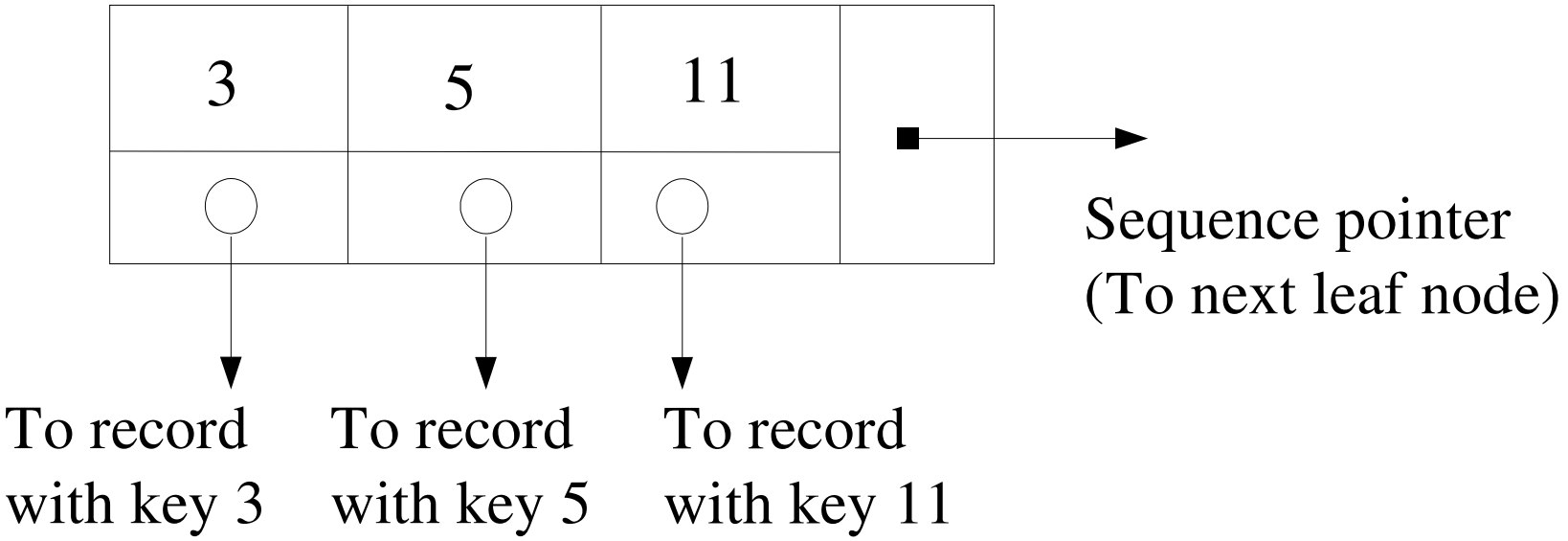
B+ trees – internal node



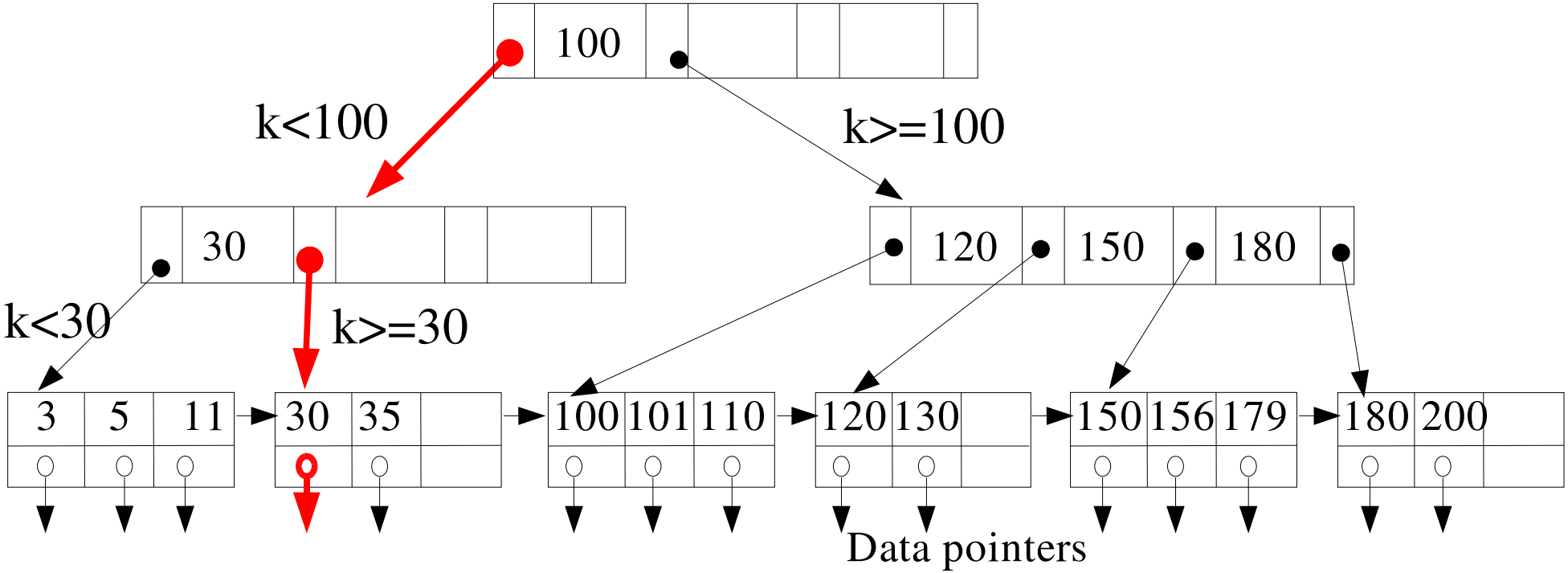
B+ tree – Leaf nodes

- Each leaf node is of the form
  - $<(K_1,Pr_1), P_2, (K_2,Pr_2), ..,(K_{q-1}, Pr_{q-1}), P_{next}>$
  - $q <= p$
  - P_{next} (sequence pointer) points to next leaf node of B+ tree
  - Pr_i is a data pointer, points to record whose key value is K_i
    - Or points to an "indirect" block of pointers to data records
      - If search key is a nonkey field
- Within each leaf node, $K_1 < K_2 < ... < K_{q-1}$
- Each leaf node has at least $\lceil p/2 \rceil$ values
- All leaf nodes are at the same level

B+ tree – leaf node



B+ tree – search



- At each level, find smallest key K_i larger than search-key
  - Follow the associated pointer (P_i)
  - If no such key found, follow last pointer in node ...until leaf node

B+ tree – built on a nonkey field

- B+ tree can be built on a search-key that is not unique
  - Many records may have the same search-key value
- Leaf nodes usually contain record pointers
- If a search value matches more than one record
  - Leaf node entry stores a pointer to an “indirect” block
    - Block contains pointers to all records with that search-key

B+ tree operations – insert

- Insert and delete are efficient but a bit complicated
  - Because nodes may overflow or underflow
- Ignoring node overflow and underflow
  - Insert data record with search-key **k**
    - Find leaf node where **k** would appear
    - If search-key **k** found
      - Add data record to file, create indirect block if there isn't one
      - Add record pointer to indirect block
    - If search-key **k** not found
      - Add data record to file
      - Insert (**k**, data pointer) in leaf node
        - Such that all search keys in leaf node remain in order

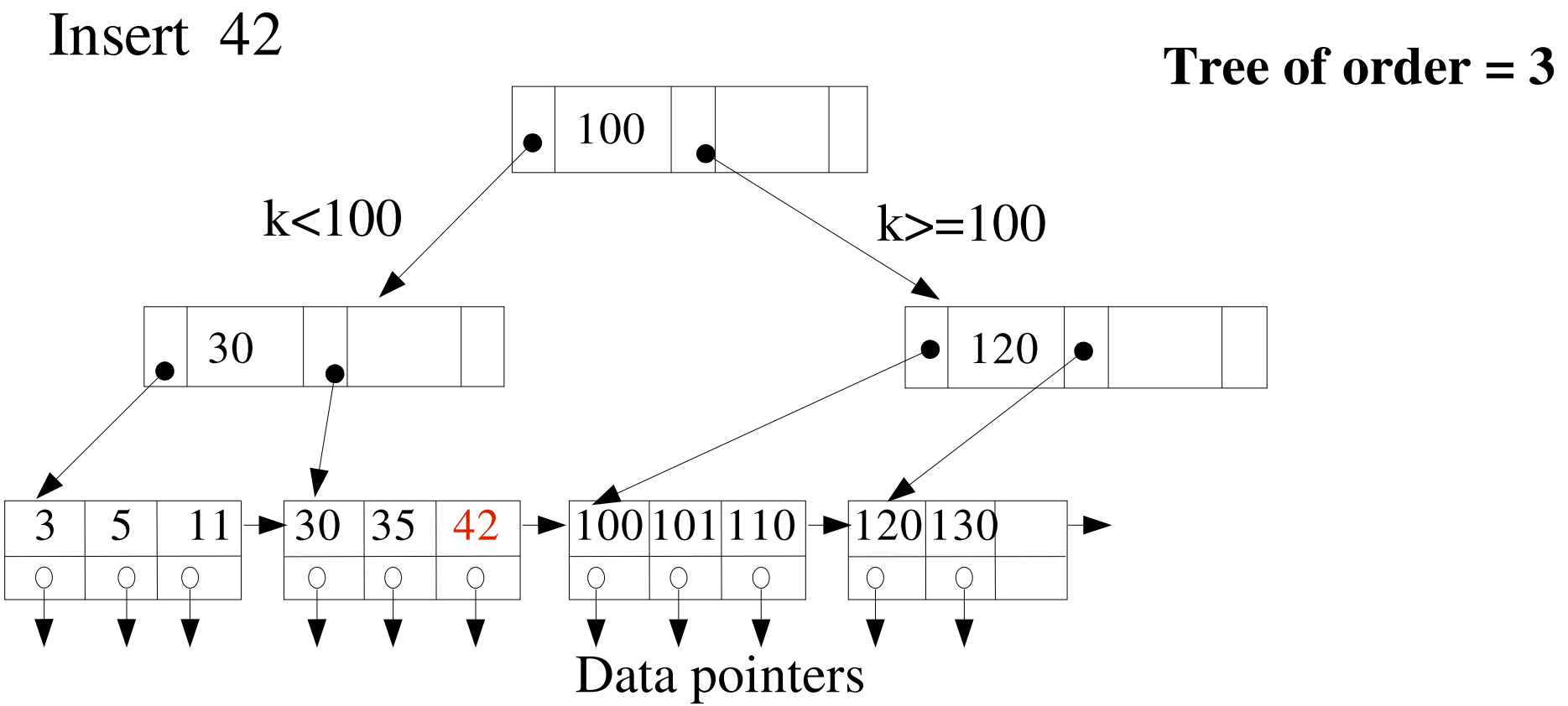
B+ tree operations – delete

- Ignoring node overflow and underflow
  - Delete data record with search-key **k**
    - Find leaf node with search-key **k**
    - Find data record pointer, delete data record from file
    - Remove (**k**, record pointer) entry from leaf node if there is no indirect block associated with that entry
      - Or if indirect block becomes empty as a result of the deletion

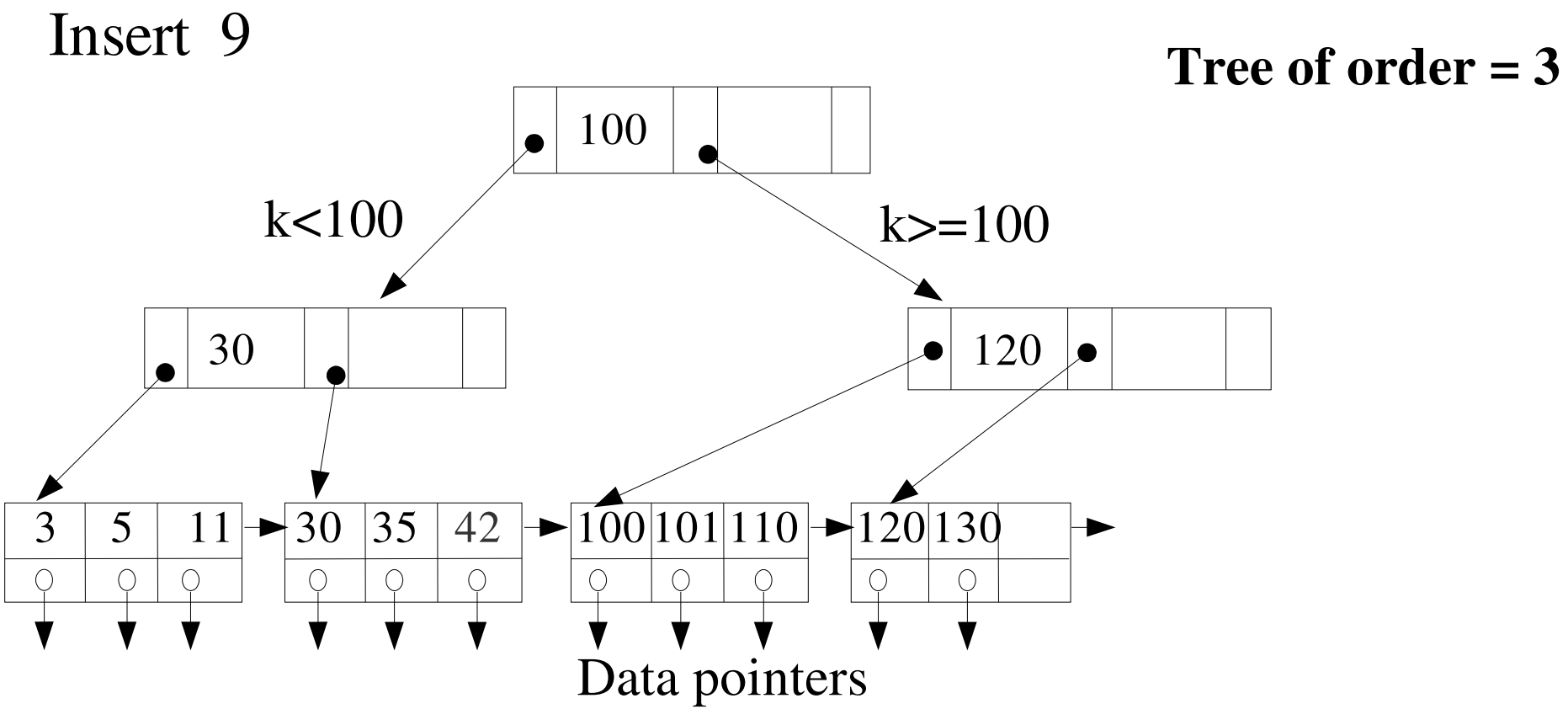
B+ insert

- Four cases
  - 1. Simple case: There is space in leaf node
  - 2. Leaf node overflow
  - 3. Internal node overflow
  - 4. New root

B+ tree insert – case 1

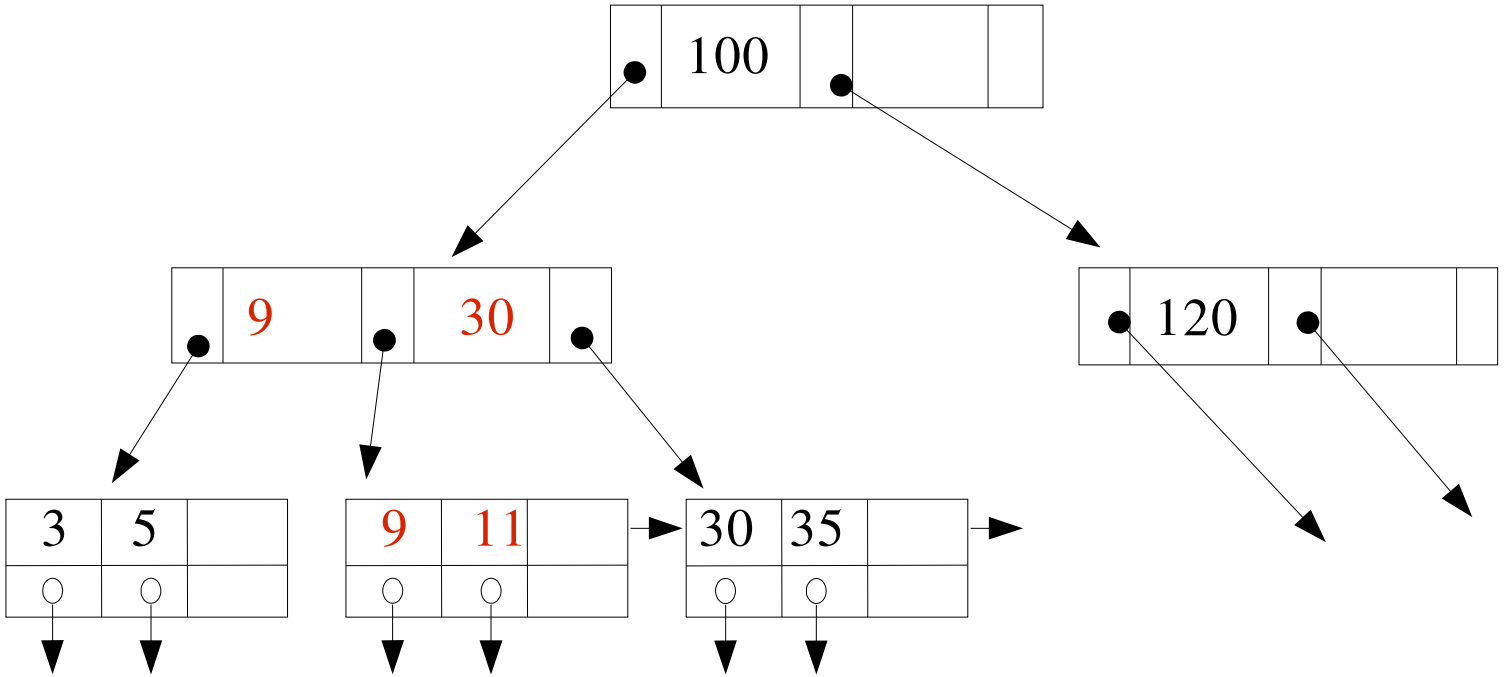


B+ tree insert – case 2



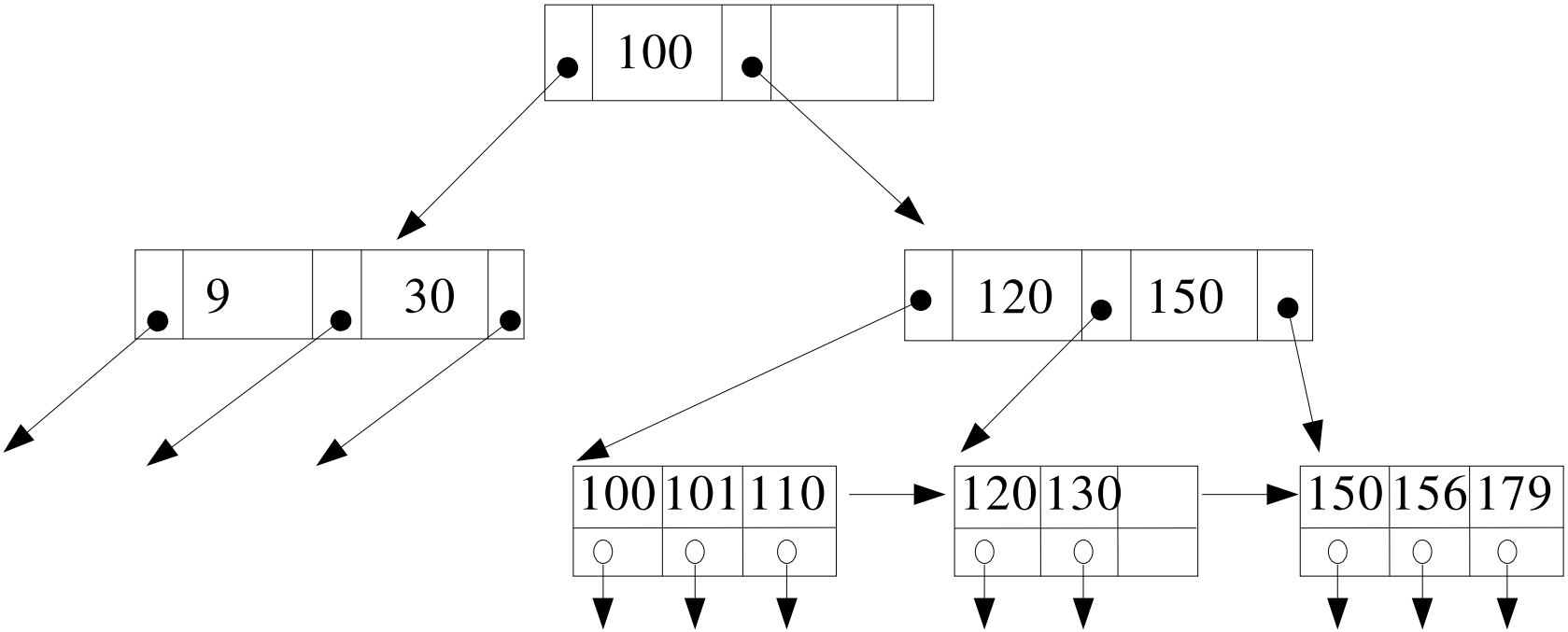
B+ tree insert – case 2 (p2)

Insert 9



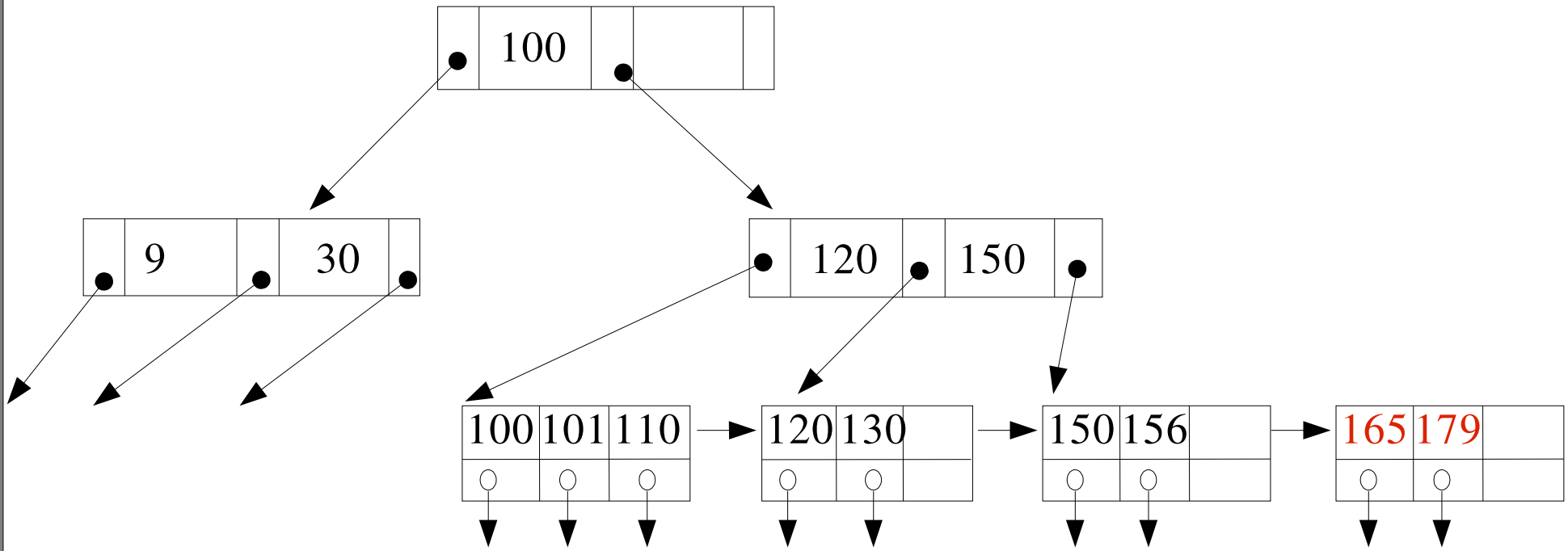
B+ tree insert – case 3

Insert 165



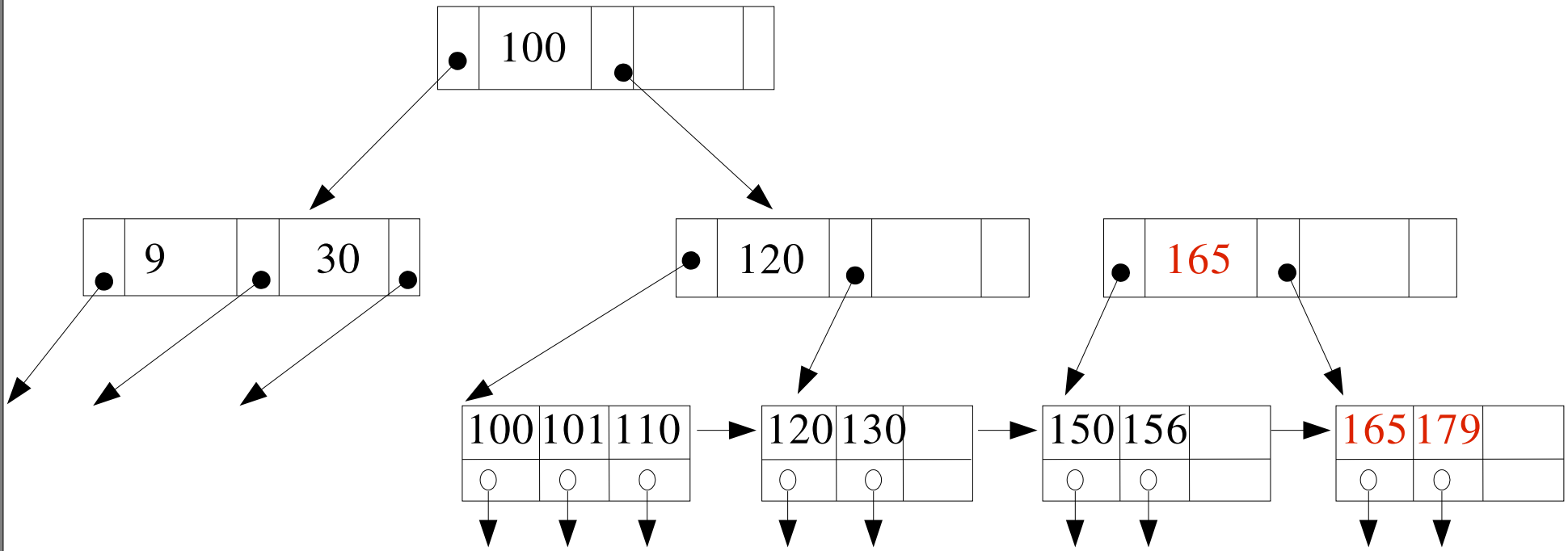
B+ tree insert – case 3 (p2)

Insert 165



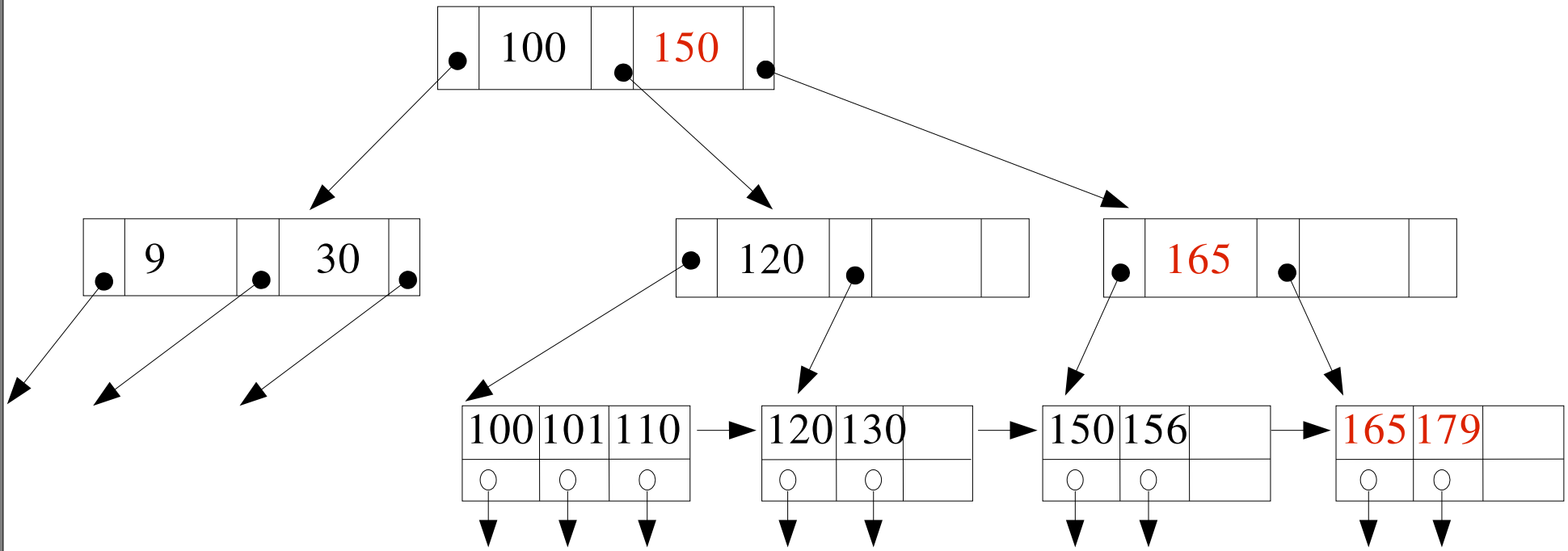
B+ tree insert – case 3 (p3)

Insert 165



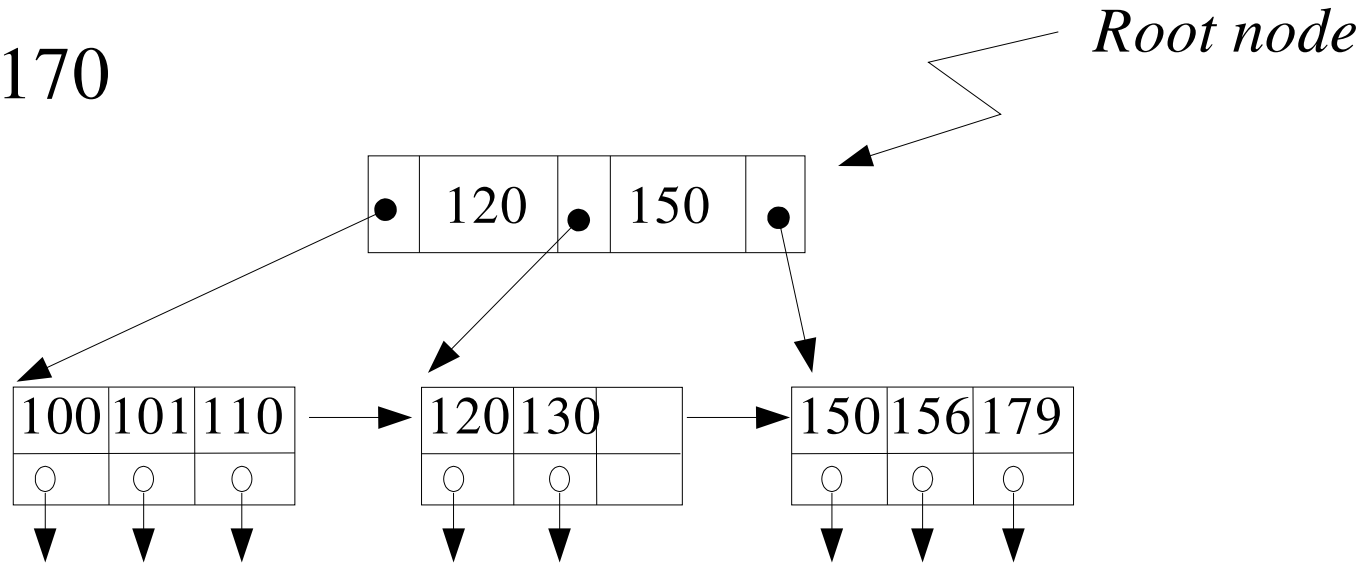
B+ tree insert – case 3 (p4)

Insert 165



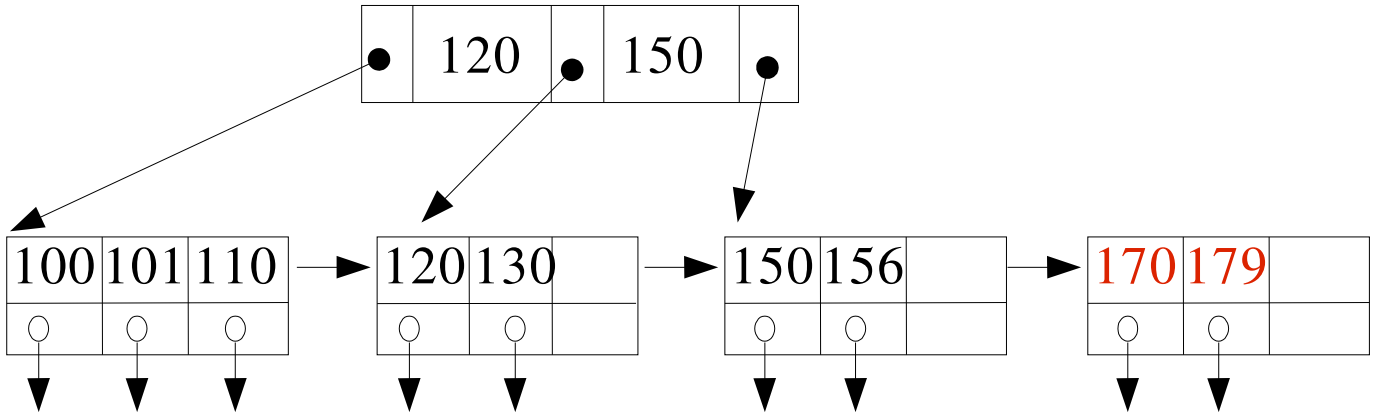
B+ tree insert – case 4 (new root)

Insert 170



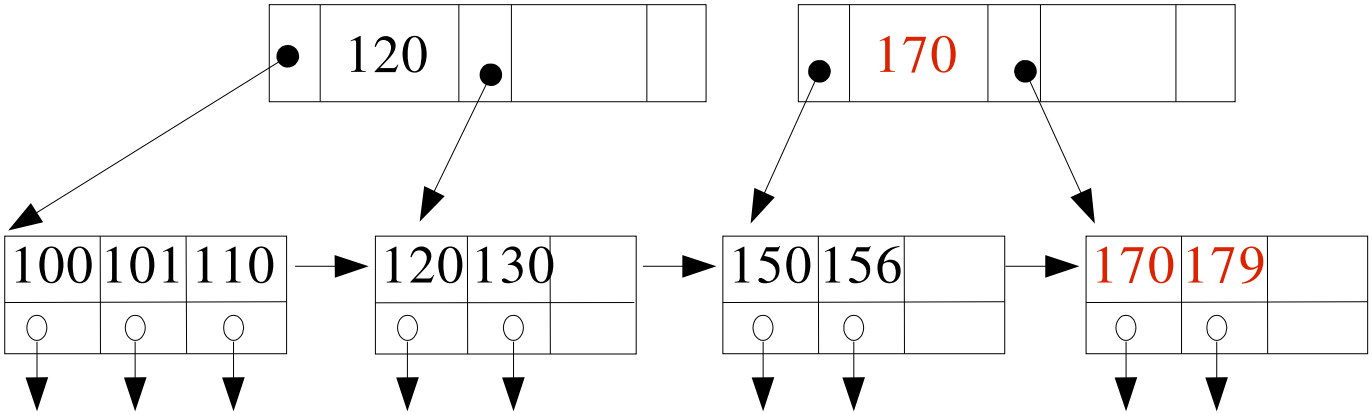
B+ tree insert – case 4 (p2)

Insert 170

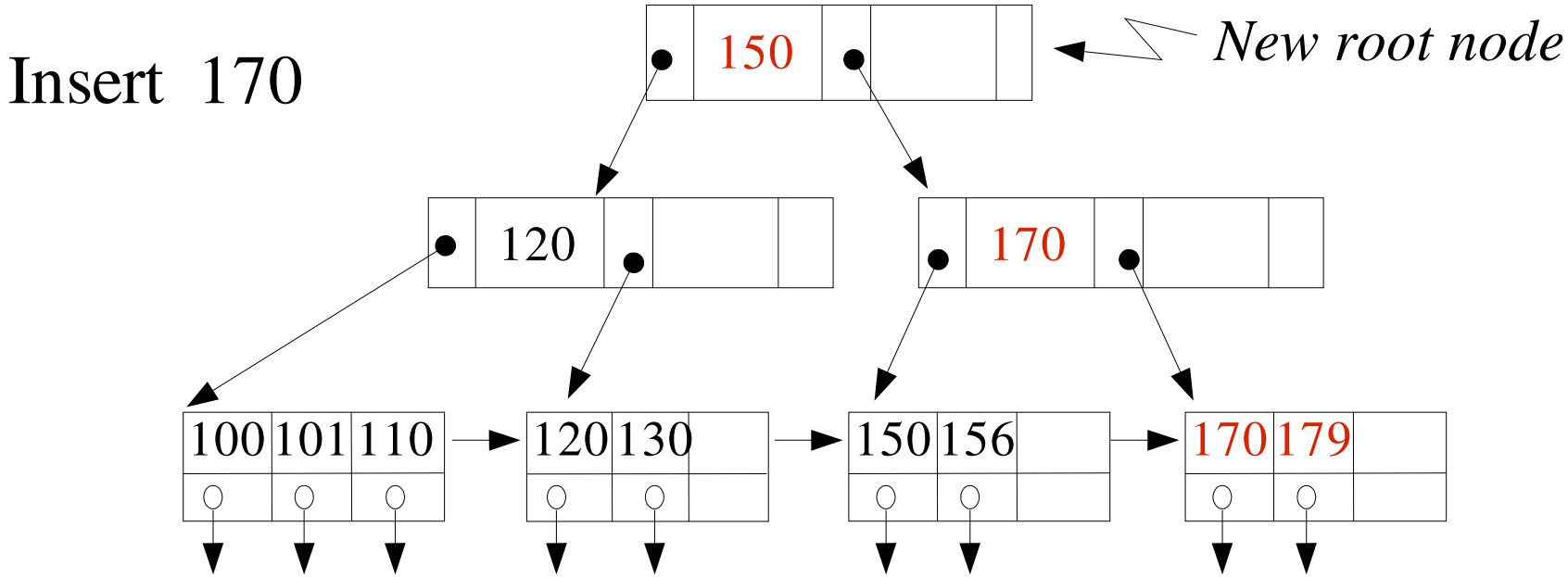


B+ tree insert – case 4 (p3)

Insert 170

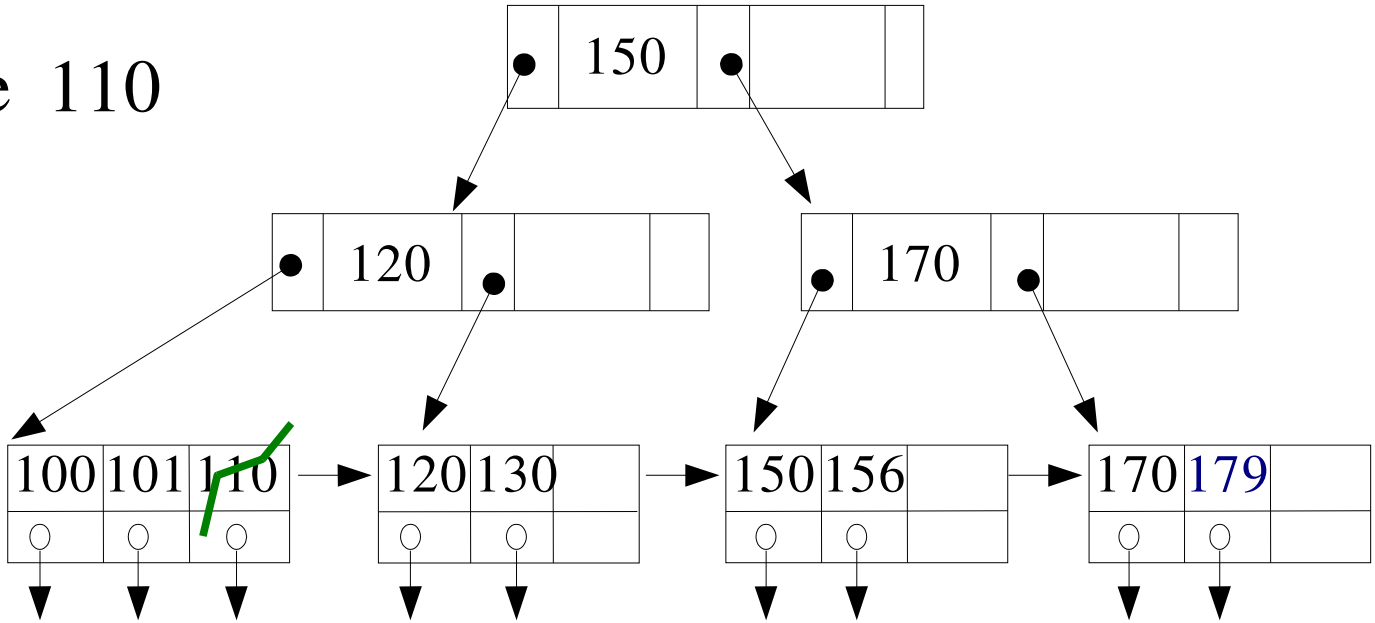


B+ tree insert – case 4 (p4)



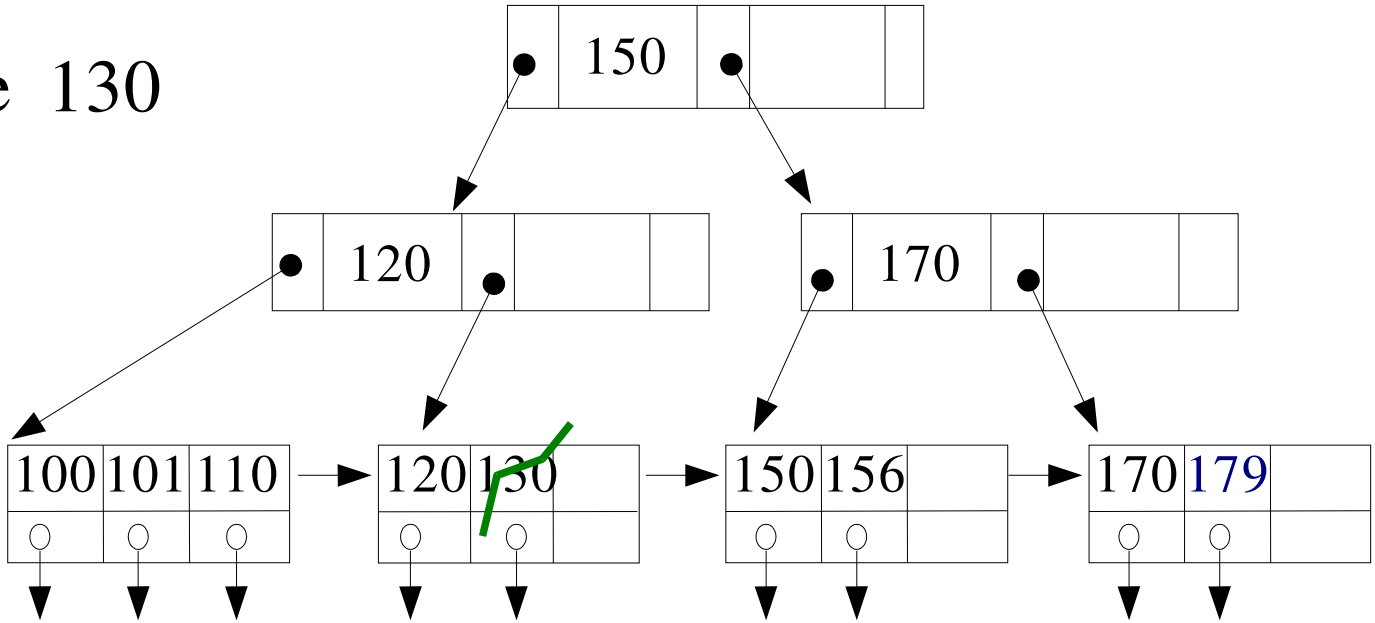
B+ tree delete

Delete 110



B+ tree delete

Delete 130



B+ tree operations – delete

- Simple case
  - Deletion does not cause underflow at leaf
- Underflow case – Key redistribution
  - Redistribute keys
    - Borrow one key from adjacent node
    - Redistribute evenly between adjacent nodes
  - Update parent nodes

B+ tree operations – delete

- Underflow case – Coalescing nodes
  - Coalesce with sibling node
  - Update pointers at parent node
  - Parent node may underflow
    - Recursively apply deletion procedure up the tree

Index summary

- B+tree, a fast and efficient multi-level index
  - Dynamic balanced data structure
  - Efficient insert and delete
    - At the expense of some space overhead
  - B+tree supports equality and range searching
- Dynamic hashing schemes
  - Dynamic schemes that grow/shrink with data file
  - Support equality search
  - But no support for range queries
- B+ trees widely implemented in commercial DBMSs

Indices in SQL

- Index optimisation not a trivial task
  - Which field(s) to create indices on?
  - In principle, the DBMS should automatically figure this out
    - Based on cost of index maintenance
    - And the benefit to query workload
  - Not quite automatic today
    - Although some DBMSs provide tools to assist
      - E.g. “index wizard”

Indices in SQL

- Index creation via SQL DDL
  - **create index** <index-name> **on** <table-name> (<attribute list>)
  - **create index** c-index **on** account (cust-city)
- If we want to declare that search-key is a candidate key
  - **create unique index** <index-name> **on** <table-name> (<attribute list>)
  - Index creation fails if table contains duplicates values for the search-key
  - Once index is created, insertion of duplicate values for that field are rejected